Yosys 0.69 PMUX 优化技术报告

Windows 原生实现

摘要

针对 RTL 中重复的分支运算和选择结构，在 Yosys 0.69 中实现了内建优化 pass。程序利用译码条件简化分支，提取可以共享的运算，再重建多输出共享的选择树。输出时，将同层且使用同一选择信号的节点合并为多位 MUX，减少综合过程中的单元数量。

在 Windows 11 上对九个用例进行完整综合，LUT 与独立 NOT 的合计数在九例中均未增加，逐例降幅平均为 20.079171%。原版和优化版导出的十八个网表均通过 Linux EQY 等价验证。八个用例首轮满足效率增幅不超过 5% 的要求，test1 按五次复核结果通过；其首轮开销仍较大，详见第 3.3 节。

第一部分　作品概述

1.1　功能与特性

程序以 Verilog RTL 为输入，在 proc 转换后识别由译码信号控制的 $mux 和 $pmux，减少重复的数据选择和分支运算。优化 pass 集成在 synth_intel 的 coarse 阶段，后续仍执行综合、ABC9 和 Cyclone IV 映射。不满足改写条件的结构保留原状，最终输出网表和资源统计。

1.2　应用领域

该方法适用于含多路数据选择、交叉开关、流水控制或反馈数据路径的 FPGA 设计。优化目标是减少映射后的逻辑资源，寄存器单独统计，不用于计算面积降幅。本作品属于 EDA 软件优化，不涉及机械结构、PCB 或独立板级实现，对应材料要求见赛题答疑 C66、C71。

1.3　主要技术特点

优化按分支条件开展。控制信号取值已知时，先化简该分支中的数据；同一控制向量对应不同译码值时，再据此判断分支互斥。运算提取同时检查位宽、符

号和信号连接关系。对于 2 至 6 位译码控制，程序将多个输出共同构建为有向无环图（DAG），并用原默认值补全缺失的编码。搜索保留节点数最少的结构，节点数相同时沿用先找到的结果。

1.4 主要性能指标

指标	测试结果
逻辑面积	九例均不增加，平均减少 20.079171%
功能	十八项等价证明通过，负对照检出反例
效率	八例首轮通过，test1 五轮复核通过
绝对资源	最大耗时 0.815048 秒，根峰值 23,453,696 字节

1.5 主要创新点

一是把译码条件用于多个优化步骤，使分支化简、运算提取和选择树重建使用一致的控制信息。

二是在加法可交换的前提下，联合评估两侧输入的选择结构，以共享 DAG 的节点数选择操作数方向。

三是利用搜索中节点数只增不减的特点提前结束无改进的试验，并将最终同层、同选择信号的节点合并为多位 MUX。

1.6 设计流程

先固定 Yosys 0.69 源码和测试输入，构建原版与优化版 Windows 程序。随后逐例执行完整综合，统计资源、耗时和内存，并导出 post-ABC 网表。最后使用 Linux EQY 比较导出网表与源 RTL，保存测试日志、证明结果和文件校验值，供复现时核对。

第二部分　系统组成及功能说明

2.1　整体介绍

系统由综合程序、性能测量脚本和形式验证脚本组成。综合及性能测试在 Windows 上完成；Linux EQY 读取 Windows 实际导出的网表，与源 RTL 进行等价比较。两部分通过导出文件衔接，图 1 给出了整体流程。

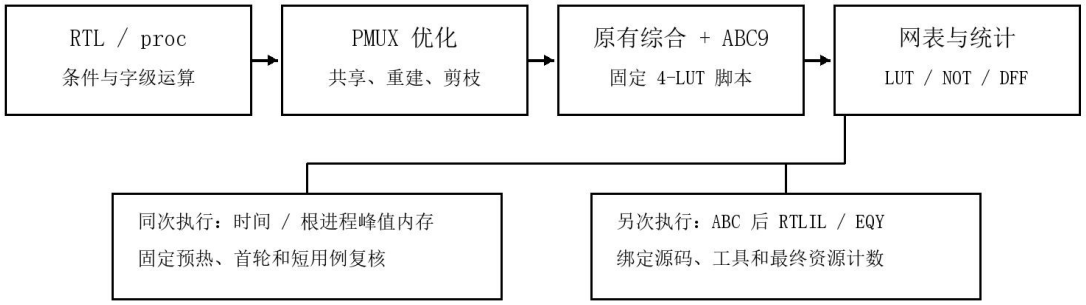


图 1　软件组成与验证流程

优化版运行 synth_intel -family cycloneiv，在 proc 后调用 pmux_competition_opt。九个用例共用同一程序和 ABC9 脚本：&dch -f; &if -K 4 -a -C 4 -F 3 -A 1，参数不随用例名称变化。原版与优化版使用同一份 ABC 二进制。本文结果对应这一固定配置，包含 PMUX 改写、统一 ABC 脚本和运行目录布局的共同影响。

2.2　硬件系统介绍

实验主机采用 x86_64 架构，综合和性能测试系统为 Windows 11，版本号 10.0.26200。Linux 环境用于运行 EQY。硬件在本项目中仅承担开发和验证工作，没有另行设计机械结构或板级电路。

2.3　软件系统介绍

2.3.1 软件整体

原版源代码为 Yosys 0.69，提交号 9f75ca1f9834a39a863915b5dae0c7b1e33533bc。两版使用 GCC 16.2.0，编译参数均包含 -Os 和 -fno-declone-ctor-dtor，标准库和 ABI 保持一致。

本次 v26 在此前完成的 v25 全量构建基础上，只重新编译 PMUX 源文件并链接，其余公共对象和原版 EXE 继续使用该次构建产物。因此，优化版程序中部分版本及时间信息继承自 v25，具体源码和二进制以包内校验值为准。交付包提供从源码完整重建两版程序的脚本；本次测试使用的是上述增量构建结果。

运行时，yosys-baseline.exe 和 yosys-candidate.exe 位于同一目录，共用 yosys-abc.exe、五个 DLL 和 share 资源。两版采用相同的 PATH 和资源目录，文件内容与对应构建产物一致。

2.3.2 软件各模块

模块与输入	主要处理
分支条件 / 比较与别名	用 SigMap 统一信号，推导分支内已知的 0、1 位
运算提取 / PMUX 及信号连接	检查互斥、位宽和使用关系，提取可共享运算
方向选择 / 合法加法分支	在预算内枚举操作数方向，减少输入选择节点
共享树 / 各输出位的数据列	枚举控制位顺序，约简并共享节点
结构改写 / 选定的共享图	通过成本检查后构造多位 $mux，替换原结构
测试 / 固定输入与两版程序	统计资源、耗时和内存，执行等价验证

（一）条件事实传播

例如 S=(ctrl==2'b10) 时，被选分支中的 ctrl[1] 和 ctrl[0] 分别为 1、0，因而可将 {ctrl[1],d,ctrl[0]} 化简为 {1'b1,d,1'b0}。这一替换只发生在对应的 B 分支，默认输入 A 和其他分支中的 ctrl 不变。$logic_not(a) 按 a==0 提取条件。比较常量含 X/Z，或同一信号位被推导出冲突取值时，不采用该条件。没有待优化目标的模块直接返回，省去后续分析。

（二）互斥分支的运算提取与保护

只有控制向量相同、译码常量不同的分支，才据此认定为互斥。例如若干分支分别计算 a^k、b^k，可将共同的 XOR 移到选择器之后。实现支持 $xor、$and、$or、$add 和 $sub，并要求 A、B、Y 的位宽及运算参数一致。提取前还要确认运算结果的各位均由该单元唯一驱动，且只被目标选择器使用，没有额外的模

块输出或控制信号用途。未知单元的连接按保守方式处理，依赖检查用于排除组合环，时序单元作为检查边界。

运算提取前增加了一个较轻量的筛选：在已选中且未标记 keep 的单元中，查找上述五类运算是否存在同类型的一对。至少两个不同的同类运算单元才有提取机会，因此没有这类单元对时可以直接结束运算提取，省去驱动和使用者索引的建立。筛选通过后，位宽、连接和依赖等检查仍照常执行；条件传播与共享树阶段不受影响。

加减法另有约束：位宽至少为 4，各分支的符号参数一致，并排除重复输入位、包含常量位的操作数和 A==B 等当前实现不接纳的结构。减法不交换操作数。运算提取会检查相关模块、线网、运算单元和目标选择器的 keep 标记；其他阶段主要检查目标单元的 keep 标记。成本估计用于决定是否尝试改写，最终是否节省资源仍由映射结果判断。

默认分支直接影响改写是否等价。共享树用原始 A 输入补全未出现的编码，不能统一补零。XOR 对整向量 X 默认值设有专门处理，AND、OR 不沿用这一规则。加减法只有在不同译码分支已经覆盖全部二进制编码时，才省略 X 默认值。形式验证按源 RTL 中 X 的无关值语义处理，因此证明结论不等同于完整的四态时序仿真。

（三）加法方向选择与预算

对于已经通过检查、且两侧操作数符号属性相同的加法，可交换某些分支的 A、B 输入，以缩小输入选择结构。程序固定第一项方向，枚举其余项；2 至 8 项加法最多检查 128 种方向组合。每种组合按自然控制位顺序构建 A、B 联合的约简 DAG，以节点数评价，不调用 ABC 试算。只有节点数严格下降、并通过原有收益门槛时才采用新的方向。

方向搜索的访问量按“方向数×2×位宽×(2×编码数−1)”估计。估计值超过 262,144 时跳过这一项搜索。这一预算仅用于加法方向搜索。

（四）共享选择树与单调下界剪枝

对于 2 至 6 位控制信号，先用原默认值补齐每个输出位的数据列，再将控制信号相同的 PMUX 放在一起搜索。控制位排列最多为 6!=720 种。每种排列用（选择位，low，high）记录节点；low==high 时直接复用子项，相同节点可由多个输出共享。数据列预先计算，模块中的实际单元在搜索结束、结构选定后统一改写。

搜索过程中，当前排列的节点数只会增加。每完成一个输出列，就将已建节点数与已有最优值比较；若当前数量已经不小于最优值，继续处理剩余列也无法改进，可提前结束这一排列。第一个排列需完整构建，之后只在节点数严格下降时更新结果。这样既减少无效搜索，也保持了节点数相同时的原有选择顺序。

选定结构通过成本检查后，v26 将单比特节点打包为多位 MUX。程序先计算拓扑深度，再按“深度、选择信号叶子 ID”分组，按原节点顺序组成 A、B、Y 向量，并为整组设置同一个 S。每个原节点仍有独立的输出位，输出解析和 PMUX 删除方式不变。

本实现按拓扑深度分组，以避免组内依赖。同组节点的输入只依赖更浅的层，不会相互依赖。如果只按选择信号分组，就无法保证这一点。打包后各位仍满足 Y=S?B:A，默认值、X/Z 和分支位置保持原含义。

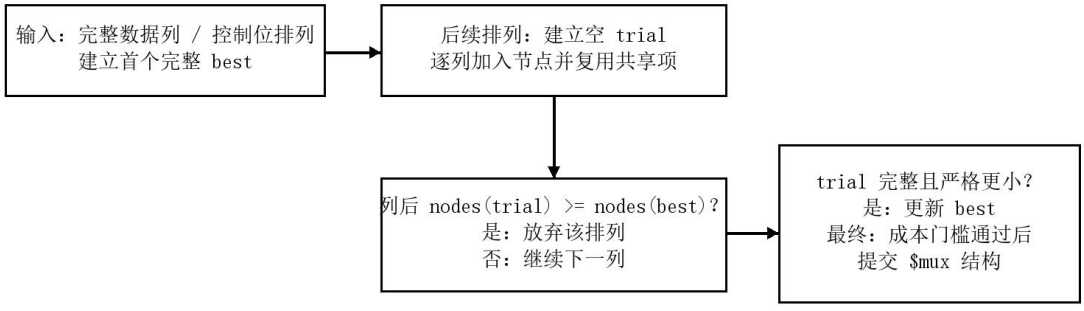


图 2　共享选择树的构建与搜索

设控制宽度为 k、输出总位宽为 W、试验节点数为 N。使用有序映射时，未剪枝搜索的时间上界可估计为 O(k!·W·2^k·log N)，数据列和试验图占用 O(W·2^k+N) 空间。实现将 k 限制在 6 以内，但输出宽度和分组数量仍会影响

开销。若最终打包为 G 组，多位 MUX 构造另需约 O(N log G) 时间和 O(N+G) 空间。该步骤减少的是 MUX 单元数，各节点的独立线网仍然保留。

第三部分　完成情况及性能参数

3.1　整体介绍

交付包包含两版 Windows Yosys 程序、共同使用的 ABC 和运行库，以及九个用例、源码、补丁、测量脚本和验证记录。本节列出该包在当前主机上的测试结果。综合及性能数据来自 Windows，等价证明针对同批 Windows 导出网表在 Linux 上完成。

3.2　工程成果

软件成果包括 PMUX 优化 pass、Windows 综合与测量入口、Linux 形式验证入口，以及 JSON 格式的资源和性能报告。源码与完整补丁可用于重新编译。项目不产生独立机械或板级电路成果。

综合报告与等价证明报告分开保存。形式证明由后续脚本完成，因此 Windows 测量报告中的 formal_status 保留为 PENDING；十八项证明的最终 PASS 状态在独立 EQY 报告中。两份报告通过 EXE、RTL 输入和网表的 SHA-256 对应到同一次测试。

3.3　特性成果

3.3.1 测试方法

综合命令为 synth_intel -family cycloneiv。按照答疑 C67、C115、C130，逻辑面积取最终网表中的 LUT 数与独立 NOT 数之和，DFF 另列。每例降幅按 r=100×(原版逻辑数−优化版逻辑数)/原版逻辑数计算，再对九例取算术平均。test3 的零降幅也计入平均值，DFF 的减少不计入逻辑面积收益。

每个用例先用原版预热 10 次，再进行首轮测量；一轮中先运行原版，再运行优化版，每次综合同时记录耗时和内存。若两侧首轮单次耗时均不足 1 秒，且任一项增幅超过 5%，则追加五轮复核。复核分别计算两版五次结果的算术平均，再求相对增幅，首轮不计入这五次平均。

本次共执行 90 次预热、18 次首轮测量和 10 次复核测量，共 118 次。首轮和全部复核记录均保留，数据没有跨版本或运行目录布局拼接。

3.3.2 九例逻辑面积

用例	原版	优化版	降幅/%	DFF 原版→优化版
test1	288	227	21.180556	222→222
test2	25	15	40.000000	0→0
test3	13	13	0.000000	17→12
test4	36	27	25.000000	10→10
case01_balanced	426	369	13.380282	12→12
case02_pipeline	450	348	22.666667	24→24
case03_crossbar	408	265	35.049020	10→10
case04_feedback	368	337	8.423913	16→16
case05_nested	413	351	15.012107	12→12

表中 case01 至 case05 的完整名称带 pmux_ 前缀。九例逻辑资源均未增加，降幅的算术平均为 20.079171455691%，达到平均减少 20% 的要求。

3.3.3 综合耗时与峰值内存

耗时采用实际经过的时间，由 time.perf_counter 在 CreateProcessW 调用前开始、根进程退出后停止，包含进程启动、完整综合、等待 ABC 和网表输出。内存通过 GetProcessMemoryInfo 读取根 Yosys 进程的 PeakWorkingSetSize，结束时再次读取峰值；该数值不包含 ABC 子进程内存。Windows Job Object 记录的子进程 CPU 数据用于辅助分析，并在测量结束时检查所有子进程已退出。实验未清空系统页缓存，也未更改系统安全配置。

首轮测试	耗时变化/%	峰值内存变化/%
test1	+132.622293	-1.698842
test2	-6.921777	+0.213904
test3	-12.993503	-1.055341

首轮测试	耗时变化/%	峰值内存变化/%
test4	-4.044307	-1.638943
case01_balanced	-8.025909	-3.315199
case02_pipeline	-5.785750	-2.635606
case03_crossbar	-29.156085	-8.967488
case04_feedback	-5.683272	-1.026393
case05_nested	-6.418096	+0.288652

表中负值表示下降。除 test1 外，其余八例首轮的耗时和根进程峰值内存增幅均不超过 5%。test1 首轮耗时增加较多，因此按第 3.3.1 节的方法追加五轮复核。

test1 首轮原版耗时 0.350374 秒，优化版为 0.815048 秒，两次合计 1.165422 秒。本报告将答疑中的“总时间不足 1 秒”按两侧单次完整综合均不足 1 秒理解。若按两次合计时间判定，test1 不满足这一复核前提；该解释尚未得到主办方的补充确认。按本文方法采用的结果如下。

采用的结果	轮数	耗时变化/%	峰值内存变化/%
test1 复核	5	-4.882056	-2.268017
test2 首轮	1	-6.921777	+0.213904
test3 首轮	1	-12.993503	-1.055341
test4 首轮	1	-4.044307	-1.638943
case01_balanced 首轮	1	-8.025909	-3.315199
case02_pipeline 首轮	1	-5.785750	-2.635606
case03_crossbar 首轮	1	-29.156085	-8.967488
case04_feedback 首轮	1	-5.683272	-1.026393
case05_nested 首轮	1	-6.418096	+0.288652

计入首轮和复核，全部优化版测量的最大耗时为 0.815047800 秒，根进程峰值内存最大为 23,453,696 字节，均低于 60 秒和 2,000,000,000 字节。本次

测试覆盖上述九例，尚不能据此判断所有不超过 1000 节点的电路，节点定义仍需按主办方口径执行。

test1 的五轮复核结果满足 5% 增幅限制，但首轮开销仍然存在。两版共用运行库和资源目录，原版预热可能使优化版受益于这些公共文件的缓存；实验没有额外预热优化版。因此，测得的时间反映了 PMUX 改写、ABC 脚本和部署方式的共同结果，不能用来单独估算某一项算法优化的加速比。

3.3.4 功能等价验证

检查项	结果
九例原版、优化版的导出网表	十八项 Linux EQY 证明通过
错误输出负对照	检出明确反例 EXPECTED_COUNTEREXAMPLE
运行平台	Windows 综合与测量，Linux 等价证明
覆盖范围	本版证明覆盖九例；历史 82 项边界测试未作为本版结果

将原版、优化版在 Windows 上实际导出的 post-ABC RTLIL 网表分别与源 RTL 比较，共完成十八项 Linux EQY 证明，全部通过。另对故意修改输出的负对照进行验证，得到明确反例；超时或 UNKNOWN 不算通过。证明对应单独导出的网表，不代表每次计时运行内部的网表都被另行导出并证明。

复现入口用文件校验值核对 Windows EXE、输入、网表和原始报告。为兼容原证明脚本，证据目录保存了两份 EXE 和一份 ABC 脚本的同内容副本，只用于文件定位，不参与 Windows 测量。Linux 工具、模型和证明策略保持固定；派生配置只更新当前 ABC 脚本的路径、校验值和 Windows 报告关联信息。配置及完整日志见交付包的证明目录。

3.3.5 复现与交付入口

将交付包完整解压到可写的英文路径，双击 run_cases.cmd，即可重新运行九例综合和性能测量。包内已有两版 EXE、共享 ABC、五个 DLL、share 资源和 Python，不需要另装 Ubuntu 或编译器。请保持目录结构完整。源码重建和 EQY 证明使用单独的入口，性能测试时不要同时执行编译或证明任务。

Windows 运行入口：run_cases.cmd

从源代码重建：source/BUILD_WINDOWS.md

Linux EQY 证明说明：PROOF_REPLAY_GUIDE.md

运行与开发依赖说明：REQUIREMENTS_WINDOWS.md

交付清单：release-status.json

以上路径均相对于交付包根目录。每次运行会建立新的结果目录；失败时保留日志，便于定位问题。源码、ABC 脚本或工具改变后，需要重新测试，原报告中的成绩不能直接沿用。

第四部分　总结

4.1　可扩展之处

下一步可改进结构成本估计，使其更接近最终 LUT 映射结果。扩大控制位宽前，还需处理排列数量快速增长的问题，并重新设置搜索预算。多位 MUX 打包也会产生遍历和分组开销，是否节省完整综合时间，应继续用相同条件下的实测结果判断。

4.2　心得体会

本项目中，局部节点数用于筛选改写方案，最终收益仍由完整综合决定。选择结构变小并不必然带来更少的 LUT；默认值、符号属性和分支互斥性也必须同时满足，才能安全共享运算。多位 MUX 保留了逐位逻辑关系，但可能影响后续映射，因此每次实现调整后都需要重新进行资源统计和等价验证。

本次交付在九例上达到平均逻辑面积减少 20% 的目标，十八项等价证明通过。效率结果按第 3.3 节列出的首轮和复核规则统计，test1 的首轮问题及规则解释仍需在现场验收中核对。

第五部分　参考文献

[1] 2026 FPGA 赛道选题指南：中科亿海微[Z]. 2026.

[2] FPGA 创新大赛问题汇总：赛题二答疑[Z]. 2026.

[3] YosysHQ. Yosys 0.69 源代码[CP].

测试记录见包内 evidence/，源码与重建说明见 source/。